**I.T. ESCUELA INDUSTRIAL SUPERIOR**
**PEDRO DOMINGO MURILLO**
**CARRERA DE INFORMATICA INDUSTRIAL**




**SIM 300 SISTEMAS MICROPROCESADOS**

**INFORME**

# SISTEMA DE CONTROL DE DISPOSITIVOS MEDIANTE BLUETOOTH CON ESP32

**ESTUDIANTE:**

1.-CORILA GUTIERREZ LUIS RODRIGO

FECHA: 12/JUNIO/2025

**LA PAZ – BOLIVIA**

El presente informe detalla el desarrollo e implementación de un sistema de control remoto basado en tecnología Bluetooth, utilizando el microcontrolador ESP32. El objetivo principal fue establecer una comunicación inalámbrica bidireccional entre una aplicación móvil y el ESP32 para gestionar de forma remota un motor de corriente continua (DC) y una luminaria de corriente alterna (AC). El sistema incorpora opciones de control mediante botones virtuales en la aplicación y reconocimiento de comandos de voz, demostrando la versatilidad del ESP32 en aplicaciones de automatización y control.

## 1. OBJETIVO GENERAL

Diseñar e implementar un sistema de control inalámbrico robusto y versátil, empleando la plataforma ESP32 y la conectividad Bluetooth, para la gestión remota de un motor DC y una luminaria AC. El sistema permitirá la interacción del usuario a través de una aplicación móvil que soporte control por interfaz gráfica (botones virtuales) y comandos de voz, sentando las bases para futuras aplicaciones de domótica y automatización.

## 2. MATERIALES Y COMPONENTES

La implementación del proyecto requirió los siguientes componentes clave:

- **Microcontrolador ESP32:** Actúa como el cerebro del sistema, proporcionando capacidad de procesamiento y conectividad Bluetooth.
- **Motor DC (5V-12V):** Un actuador para demostrar el control de velocidad o dirección (dependiendo de la implementación específica).
- **Módulo Relé de 5V:** Permite el control de cargas de AC, como el foco, de forma segura y aislada del ESP32.
- **Transistor NPN 2N2222:** Utilizado como interruptor electrónico para conmutar la corriente del relé, protegiendo los pines GPIO del ESP32.
- **Diodo Rectificador 1N4007:** Empleado para proteger el circuito de los picos de voltaje inversos generados por la bobina del relé.
- **Fuentes de Alimentación (5V y 12V):** Proporcionan la energía necesaria para el ESP32, el relé y el motor DC, respectivamente.
- **Cables Dupont y Protoboard:** Para el prototipado y la interconexión de los componentes.
- **Aplicación Móvil Compatible con Bluetooth Serial:** Aplicaciones como "Bluetooth Terminal" o "Serial Bluetooth" fueron utilizadas para enviar comandos y recibir datos del ESP32.

## 3. FUNDAMENTO TEÓRICO

### 3.1 Conectividad Bluetooth del ESP32

El ESP32 es un microcontrolador altamente integrado que se distingue por sus capacidades de conectividad inalámbrica, incluyendo Wi-Fi y Bluetooth. Para este proyecto, se explotaron sus funcionalidades de **Bluetooth Clásico (BR/EDR)**, que permiten una comunicación serial (UART) con dispositivos móviles. Esta comunicación se establece mediante un perfil de puerto serie (SPP), facilitando el intercambio de datos entre la aplicación móvil y el microcontrolador. La librería BluetoothSerial.h de Arduino para ESP32 simplifica enormemente la gestión de esta comunicación.

### 3.2 Principios de Conmutación con Transistores y Relés

- **Transistor 2N2222:** Este transistor NPN opera como un interruptor de estado sólido. Al aplicar una pequeña corriente a su base (mediante un pin GPIO del ESP32), se permite el flujo de una corriente mucho mayor entre su colector y emisor. Esto es crucial para proteger el ESP32, ya que evita que la carga directa del relé o motor sobrecargue sus pines de salida.
- **Módulo Relé:** Un relé es un interruptor electromecánico que permite controlar cargas de alta potencia (como focos AC) con una señal de baja potencia (del ESP32 a través del transistor). La bobina del relé, al ser energizada, crea un campo magnético que cierra o abre sus contactos. El uso de los contactos **Normalmente Abiertos (NO)** y **Común (COM)** es fundamental para que el foco se encienda cuando el relé es activado.
- **Diodo 1N4007:** La bobina de un relé es una carga inductiva. Al desenergizarse, puede generar un pico de voltaje inverso ("back-EMF") que podría dañar el transistor o el ESP32. El diodo 1N4007, conectado en paralelo a la bobina del relé en polarización inversa, actúa como un "diodo de volante" o "flyback diode", proporcionando un camino para que esta energía inductiva se disipe de forma segura.

## 4. DESARROLLO DEL PROYECTO

El proyecto se estructuró en las siguientes etapas principales:

1. **Configuración de la Comunicación Bluetooth Serial:** Se inicializó la comunicación Bluetooth en el ESP32 utilizando la librería BluetoothSerial.h. Se estableció un nombre de dispositivo Bluetooth para el ESP32, lo que facilita su emparejamiento con el dispositivo móvil. El ESP32 se configuró para escuchar y recibir comandos textuales a través del puerto serial Bluetooth. Por ejemplo, la recepción de un carácter '1' podría interpretarse como el comando para encender el foco, y un '2' para apagarlo.
2. **Control del Motor DC:** La activación del motor DC se implementó directamente a través de un transistor 2N2222. Un pin GPIO del ESP32 se configuró para controlar la base del transistor, permitiendo o interrumpiendo el flujo de corriente hacia el motor desde una fuente de alimentación externa de 12V. La lógica de control en el ESP32 se encargó de interpretar los comandos Bluetooth pertinentes para activar o desactivar el motor.
3. **Control del Foco AC:** Para el foco AC, se utilizó un módulo relé de 5V. El ESP32 envía una señal de bajo nivel (HIGH/LOW) a la base del transistor 2N2222, que a su vez activa la bobina del relé. Los contactos del relé (Normalmente Abierto y Común) se interconectaron con la fuente de alimentación AC y el foco, de tal manera que el foco se enciende cuando el relé es energizado.
4. **Implementación de Comandos de Voz:** La integración de comandos de voz se realizó a nivel de la aplicación móvil. La aplicación, al utilizar su función de reconocimiento de voz, convierte las palabras habladas en texto. Este texto es luego enviado al ESP32 a través de la comunicación Bluetooth serial. En el firmware del ESP32, se implementó una lógica de "parsing" para interpretar estas cadenas de texto (ej., "encender luz", "apagar motor") y ejecutar las acciones correspondientes.

**5. CONEXIONES ELÉCTRICAS**

A continuación, se detalla el esquema de conexiones eléctricas clave para cada componente:

- **Motor DC:**
    - Terminal positivo del motor a la fuente de alimentación de 12V.
    - Terminal negativo del motor al colector del transistor 2N2222.
    - Emisor del 2N2222 a tierra (GND común).
- **Foco AC:**
    - Un cable de la alimentación AC al contacto **COM** (Común) del módulo relé.
    - El otro contacto del foco AC al contacto **NO** (Normalmente Abierto) del módulo relé.
    - El otro cable de la alimentación AC directamente al foco.
- **Módulo Relé:**
    - Pin **IN** del relé al colector del transistor 2N2222 (o directamente a un pin GPIO del ESP32 si no se usa transistor de control separado para la bobina, aunque no recomendado para carga directa).
    - Pin **VCC** del relé a la fuente de alimentación de 5V.
    - Pin **GND** del relé a tierra (GND común).
- **Transistor 2N2222 (Control de Relé/Motor):**
    - Base del 2N2222 a un pin GPIO del ESP32 (ej. GPIO23) a través de una **resistencia limitadora de corriente de 1kΩ**.
    - Colector del 2N2222 a la carga (bobina del relé o motor).
    - Emisor del 2N2222 a tierra (GND común).
- **Diodo 1N4007:**
    - Conectado en paralelo con la bobina del relé, con el **cátodo (banda)** hacia el pin de alimentación positiva del relé (VCC) y el **ánodo** hacia el pin de control (IN) del relé.

**6. RESULTADOS OBTENIDOS**

El desarrollo del sistema de control remoto fue exitoso, cumpliendo con los objetivos planteados:

Este proyecto demostró que controlar dispositivos cotidianos con un ESP32 y Bluetooth es sorprendentemente sencillo y efectivo. Pudimos encender y apagar un foco y un motor no solo con toques en el celular, sino también con nuestra propia voz, lo que hizo la interacción mucho más natural.

**I.T. ESCUELA INDUSTRIAL SUPERIOR**
**PEDRO DOMINGO MURILLO**
**CARRERA DE INFORMATICA INDUSTRIAL**




**SIM 300 SISTEMAS MICROPROCESADOS**

**INFORME**

# SISTEMA DE CONTROL Y MONITOREO LOCAL BASADO EN ESP32 Y WI-FI

**ESTUDIANTE:**

1.-CORILA GUTIERREZ LUIS RODRIGO

FECHA: 12/JUNIO/2025

**LA PAZ – BOLIVIA**

El presente informe detalla la implementación de un sistema de control y monitoreo remoto utilizando el microcontrolador ESP32 como servidor web embebido. El objetivo principal fue desarrollar una interfaz de usuario accesible vía Wi-Fi desde cualquier navegador web en la red local, permitiendo la manipulación de un motor de corriente continua (DC), un foco de corriente alterna (AC) y un LED, así como la visualización en tiempo real del valor de un potenciómetro lineal. Este proyecto demuestra la capacidad del ESP32 para crear soluciones de Internet de las Cosas (IoT) locales sin depender de infraestructura en la nube externa.

**1. OBJETIVO GENERAL**

Implementar una interfaz de control y monitoreo en tiempo real, accesible a través de una red Wi-Fi local, utilizando el microcontrolador ESP32 como servidor web. Esta interfaz permitirá la activación y desactivación de diversos actuadores (motor DC, foco AC, LED) y la visualización dinámica del estado de un sensor analógico (potenciómetro), consolidando al ESP32 como una plataforma versátil para aplicaciones de automatización y adquisición de datos en entornos locales.

**2. MATERIALES Y COMPONENTES**

Para la realización de este proyecto, se utilizaron los siguientes componentes:

- **Microcontrolador ESP32:** Con capacidad de conectividad Wi-Fi, sirviendo como servidor web y controlador principal.
- **Motor DC:** Actuador para demostración de control de movimiento.
- **Módulo Relé de 5V:** Esencial para conmutar el foco AC de manera segura, aislando el circuito de control del ESP32 de la alta tensión.
- **Diodo Emisor de Luz (LED):** Indicador visual simple para pruebas de control digital.
- **Potenciómetro Lineal (10kΩ):** Sensor analógico para demostrar la capacidad de lectura y visualización de datos en tiempo real.
- **Resistencias (220Ω, 1kΩ):** Para limitar la corriente en el LED y en las bases de los transistores.
- **Transistores NPN (2N2222 o similares):** Utilizados como interruptores para activar el relé y el motor DC, protegiendo los pines del ESP32.
- **Cables Dupont, Protoboard:** Para conexiones eléctricas y prototipado.
- **Fuente de Alimentación:** Proporciona la energía necesaria para el ESP32 y los actuadores.

**3. FUNDAMENTO TEÓRICO**

**3.1 Servidor Web Embebido en ESP32**

El ESP32, gracias a su potente procesador y su pila de red Wi-Fi, puede configurarse para operar como un **servidor web HTTP**. Esto significa que puede hospedar y servir páginas web directamente desde su memoria interna a cualquier cliente (como un navegador web en un smartphone o computadora) conectado a la misma red local.

- **HTML (HyperText Markup Language):** Define la estructura y el contenido de la página web.
- **CSS (Cascading Style Sheets):** Se utiliza para estilizar y presentar visualmente la página web.

- **JavaScript:** Es crucial para la interactividad de la página. Permite la creación de botones dinámicos y, lo más importante, la actualización asincrónica de datos sin recargar la página completa.

**3.2 Comunicación Asincrónica con AJAX**

La visualización en tiempo real del valor del potenciómetro se logra mediante **AJAX (Asynchronous JavaScript and XML)**. AJAX permite que una página web realice solicitudes HTTP al servidor (ESP32) en segundo plano, sin interrumpir la experiencia del usuario o requerir una recarga completa de la página. El ESP32 responde a estas solicitudes con los datos solicitados (ej. el valor del ADC), que luego son actualizados dinámicamente en la interfaz web por JavaScript. Esto crea una sensación de "tiempo real" y una experiencia de usuario fluida.

**3.3 Adquisición de Datos Analógicos (ADC)**

El ESP32 incorpora convertidores analógico-digitales (ADC), que transforman una señal de voltaje analógica (como la proveniente de un potenciómetro) en un valor digital que el microcontrolador puede procesar. El pin **GPIO36** es parte del ADC1 del ESP32 y es ideal para este propósito, ya que no tiene pull-ups internos que puedan afectar la lectura.

**3.4 Control de Actuadores con Transistores y Relés**

Similar al informe anterior, se utilizan **transistores NPN (2N2222)** como interruptores electrónicos para permitir que los pines de baja corriente del ESP32 controlen cargas que requieren mayor corriente o voltaje (como el relé y el motor DC). El **módulo relé** es fundamental para el control seguro del foco AC, proporcionando aislamiento galvánico entre el circuito de baja tensión del ESP32 y la red eléctrica.

**4. DESARROLLO DEL PROYECTO**

El desarrollo del sistema se llevó a cabo en las siguientes fases:

1. **Configuración del Servidor Web Wi-Fi:**
    - El ESP32 se configuró para conectarse a una red Wi-Fi local existente (modo estación) o para crear su propia red (modo Access Point), optándose por la primera para integración en la red doméstica.
    - Se inicializó la instancia del servidor web (WebServer o ESPAsyncWebServer para un rendimiento más robusto) y se definieron los manejadores de rutas (endpoints) para las solicitudes HTTP GET. Por ejemplo, http://[IP_ESP32]/motor_on o http://[IP_ESP32]/led_toggle.
    - La página HTML que contiene la interfaz de usuario (botones, área de visualización del potenciómetro) se alojó directamente en el código del ESP32 o en el sistema de archivos SPIFFS/LittleFS del ESP32 para páginas más grandes.
2. **Implementación de la Interfaz Web (HTML, CSS, JavaScript):**
    - Se diseñó una página HTML simple con botones (<button>) para cada actuador (motor DC, LED, foco AC).
    - Se utilizó JavaScript para asociar eventos onclick a estos botones, los cuales enviaban solicitudes GET al ESP32 al ser presionados.

- Se implementó una sección para mostrar el valor del potenciómetro, actualizada periódicamente mediante solicitudes AJAX. Una función JavaScript realizaba una solicitud GET a un endpoint específico (ej. /pot_value) en el ESP32, y al recibir la respuesta (el valor numérico), actualizaba el contenido de un elemento HTML ( o ).

3. **Control de Actuadores:**
    - Dentro del firmware del ESP32, los manejadores de rutas HTTP recibieron las solicitudes de los botones.
    - Al detectar una solicitud como /motor_on, el código del ESP32 activó el pin GPIO correspondiente que controla el transistor del motor. De manera similar, se implementó la lógica para encender/apagar el LED y el foco a través del relé.
    - Se configuraron los pines GPIO de salida correspondientes a cada actuador.
4. **Lectura y Visualización del Potenciómetro:**
    - El potenciómetro se conectó al pin **GPIO36**, configurado como entrada analógica.
    - En el código del ESP32, se realizó una lectura analógica del potenciómetro utilizando analogRead().
    - El manejador de la ruta /pot_value (o similar) leía este valor y lo enviaba como respuesta a la solicitud AJAX desde el navegador, generalmente como un valor numérico simple o en formato JSON.

**5. CONEXIONES ELÉCTRICAS**

- **Potenciómetro (10kΩ):**
    - Pin central a **GPIO36 (ADC1_CH0)** del ESP32.
    - Un extremo a **3.3V** del ESP32.
    - El otro extremo a **GND** del ESP32.
- **LED:**
    - Ánodo (pata larga) a una **resistencia de 220Ω**.
    - El otro extremo de la resistencia a un pin GPIO digital del ESP32 (ej. GPIO2).
    - Cátodo (pata corta) a **GND** del ESP32.
- **Motor DC y Foco AC:**
    - Las conexiones para el **motor DC** (controlado por un transistor 2N2222) y el **foco AC** (controlado por un módulo relé de 5V), incluyendo los diodos de protección, se realizaron de forma idéntica a las descritas en el **Informe 1: Control de ESP32 mediante Bluetooth**, con la diferencia de que los pines GPIO de control ahora son activados por la lógica del servidor web en lugar de comandos seriales.

## 6. RESULTADOS OBTENIDOS

La implementación del sistema de control y monitoreo vía Wi-Fi fue altamente exitosa y cumplió con todos los objetivos planteados:

- **Control Remoto Robusto:** Se logró un control completo y fiable de los actuadores (motor DC, LED, foco AC) desde cualquier dispositivo conectado a la misma red Wi-Fi, utilizando un navegador web estándar como interfaz de usuario.
- **Interfaz de Usuario Intuitiva:** La página web generada por el ESP32 fue responsiva y fácil de operar, proporcionando una experiencia de usuario clara y directa.
- **Monitorización en Tiempo Real:** La lectura del potenciómetro fue mostrada dinámicamente en la interfaz web con actualizaciones en tiempo real (o con una latencia mínima), demostrando la eficacia de la implementación AJAX para la adquisición y visualización de datos de sensores.
- **Estabilidad del Servidor Web:** El servidor web embebido en el ESP32 demostró ser estable y capaz de manejar múltiples solicitudes sin interrupciones, incluso durante actualizaciones constantes del sensor.

## 7. CONCLUSIONES Y PERSPECTIVAS FUTURAS

Este proyecto con el ESP32 y Wi-Fi nos mostró lo **fácil que es tener el control en nuestras manos, literalmente desde cualquier navegador web**. Pudiendo manejar un motor, un foco y un LED, e incluso ver cómo cambiaba el valor de un sensor en tiempo real, demostramos que no necesitamos grandes sistemas ni la nube para tener automatización en casa.

Lo más valioso de esto es la **independencia que nos da**. Podemos crear sistemas inteligentes que funcionan en nuestra propia red, sin depender de servicios externos. Es como tener nuestro propio cerebro domótico personal. Esto abre un mundo de posibilidades para la automatización del hogar que es más **privada, directa y hecha a nuestra medida**. Es un gran paso para cualquiera que quiera empezar a construir su propio hogar conectado.

**I.T. ESCUELA INDUSTRIAL SUPERIOR**

**PEDRO DOMINGO MURILLO**

**CARRERA DE INFORMATICA INDUSTRIAL**




# SIM 300 SISTEMAS MICROPROCESADOS

## INFORME

## SERVOMOTORES Y MOTORES DC/AC CON ESP32

**ESTUDIANTE:**

1.-CORILA GUTIERREZ LUIS RODRIGO

FECHA: 12/JUNIO/2025

**LA PAZ – BOLIVIA**

El presente informe documenta la implementación de diversas estrategias de control para servomotores y motores de corriente continua (DC), además del control de cargas de corriente alterna (AC) utilizando un triac, todo ello gestionado por el microcontrolador ESP32. El proyecto se subdividió en cuatro experimentos clave, abordando el control por interacción física (botones) y por lógica temporizada. Los resultados obtenidos demuestran la alta versatilidad y capacidad del ESP32 para manejar múltiples tipos de actuadores, lo que lo posiciona como una plataforma ideal para proyectos de robótica, automatización y control de potencia.

## 1. OBJETIVO GENERAL

Diseñar e implementar soluciones robustas para el control de servomotores y motores DC/AC utilizando el microcontrolador ESP32, explorando métodos de activación por medio de entradas físicas (pulsadores) y por temporización programada. El objetivo es consolidar el conocimiento sobre el manejo de señales PWM para servos, la inversión de giro para motores DC y la conmutación de cargas AC, sentando las bases para sistemas de control de movimiento y potencia más complejos.

## 2. MATERIALES Y COMPONENTES

Para la realización de los experimentos, se emplearon los siguientes componentes:

- **ESP32 DevKit:** Plataforma principal de procesamiento y control, con capacidad para generar múltiples señales PWM y manejar entradas/salidas digitales.
- **2 Servomotores SG90:** Servos de bajo costo, ideales para demostraciones de control de posición angular mediante PWM.
- **3 Botones Físicos (Pulsadores Momentáneos):** Para la interacción manual y el control de los actuadores.
- **Motor DC:** Motor de corriente continua para demostrar la inversión del sentido de giro.
- **Triac (BT136 o similar) + Optoacoplador MOC3021:** Componentes clave para el control seguro y aislado de cargas de corriente alterna.
- **Resistencias (220Ω, 1kΩ):** Para limitar la corriente en LEDs (si se usan indicadores) y en las bases de transistores o los LEDs internos de optoacopladores.
- **Fuentes de Alimentación (5V y 12V):** Para energizar el ESP32, los servos y el motor DC, respectivamente.
- **Cables Dupont, Protoboard:** Elementos esenciales para el prototipado y la interconexión de componentes.

## 3. FUNDAMENTO TEÓRICO

### 3.1 Control de Servomotores mediante PWM

Los **servomotores** son actuadores que permiten un control preciso de la posición angular. Se controlan mediante una señal de **Modulación por Ancho de Pulso (PWM)**. La frecuencia de esta señal suele ser de 50 Hz (un pulso cada 20 ms), y el ángulo del servo se determina por la **duración del pulso (ancho de pulso)**:

- Normalmente, un pulso de aproximadamente 1 ms corresponde a 0°.
- Un pulso de 1.5 ms a 90°.
- Un pulso de 2 ms a 180°.

El ESP32 es excepcionalmente apto para esto, ya que posee múltiples **canales de PWM** hardware (16 canales independientes) que pueden ser configurados con alta precisión para generar estas señales, liberando al procesador de la carga de temporización manual.

**3.2 Control de Motores DC**

Los motores de corriente continua (DC) giran en un sentido u otro dependiendo de la polaridad de la tensión aplicada. Para invertir su giro desde un microcontrolador, se suele emplear un **Puente H** (compuesto por transistores o un circuito integrado dedicado como el L298N). Este circuito permite cambiar la dirección de la corriente que fluye a través del motor sin cambiar la polaridad de la fuente de alimentación. La lógica temporizada para el cambio de sentido puede implementarse usando la función millis(), que permite ejecutar acciones a intervalos regulares sin bloquear la ejecución del resto del código.

**3.3 Control de Cargas AC con Triac y Optoacoplador**

Para controlar cargas de corriente alterna (como un motor AC o una luminaria) con un microcontrolador de baja tensión como el ESP32, es fundamental utilizar un circuito de interfaz que proporcione **aislamiento galvánico** y soporte la alta tensión/corriente de la AC.

- **Optoacoplador MOC3021 (u otros de cruce por cero/no cruce por cero):** Contiene un LED infrarrojo y un fototriac en un solo encapsulado. Al encender el LED (mediante una señal digital del ESP32), el fototriac interno se activa, disparando el triac principal. Esto proporciona aislamiento total entre el circuito de control del ESP32 y el circuito de AC, lo que es crucial para la seguridad.
- **Triac:** Es un semiconductor que puede conducir corriente en ambas direcciones (AC) una vez que ha sido disparado. Una vez que el triac se enciende, permanece conduciendo hasta que la corriente de AC cae por debajo de un umbral mínimo (cruce por cero en la onda senoidal). Esto lo hace ideal para conmutar cargas AC.

**4. DESARROLLO DE LOS PROYECTOS**

El desarrollo se articuló en cuatro proyectos interconectados, cada uno explorando un aspecto específico del control de actuadores:

1. **Primer Proyecto: Control Básico de Servo por Botón**
    - **Funcionalidad:** Un servomotor SG90 se conectó a un pin PWM del ESP32. Un botón físico, configurado con una resistencia pull-down, se conectó a un pin GPIO de entrada.
    - **Lógica:** Al presionar el botón, el servo se movía a un ángulo predefinido (ej., 90°). Al soltar el botón, el servo regresaba a su posición inicial (ej., 0°). Esto demostró la capacidad del ESP32 para responder a entradas físicas y controlar servos.
2. **Segundo Proyecto: Control de Servo con Posiciones Múltiples**
    - **Funcionalidad:** Se utilizaron dos botones para controlar un único servomotor.

- **Lógica:** Un botón movía el servo a una posición extrema (ej., 180°), mientras que el otro botón lo regresaba a la otra posición extrema (ej., 0°). Esto permitió explorar un control más versátil de la posición.

3. **Tercer Proyecto: Inversión Automática de Giro de Motor DC**
    - **Funcionalidad:** Un motor DC se conectó al ESP32 a través de un circuito de Puente H (o transistores cruzados).
    - **Lógica:** El ESP32 se programó para invertir el sentido de giro del motor DC automáticamente cada 5 segundos. Se utilizó la función millis() para implementar esta temporización no bloqueante, lo que permitió que el microcontrolador realizara otras tareas mientras esperaba el intervalo de tiempo.
4. **Cuarto Proyecto: Control de Motor AC con Triac**
    - **Funcionalidad:** Un motor de corriente alterna (o una carga AC equivalente) se conectó a la red eléctrica a través de un circuito de triac y optoacoplador MOC3021. Una señal digital del ESP32 controlaba la activación del optoacoplador.
    - **Lógica:** Una señal (por ejemplo, desde un botón virtual o un comando de temporización) del ESP32 activaba el optoacoplador, que a su vez disparaba el triac, encendiendo el motor AC. La desactivación de la señal del ESP32 apagaba el motor. Este proyecto es crucial para aplicaciones de domótica que requieren el control de electrodomésticos o luminarias de AC.

**5. CONEXIONES CLAVE**

- **Servomotores SG90:**
    - Cable de Señal (Amarillo/Naranja) a pines GPIO del ESP32 configurados como PWM (ej., GPIO13, GPIO14).
    - Cable VCC (Rojo) a fuente de 5V.
    - Cable GND (Marrón/Negro) a GND común.
- **Botones Físicos:**
    - Un terminal del botón a un pin GPIO del ESP32 (configurado como INPUT_PULLUP o con resistencia pull-down externa).
    - El otro terminal del botón a GND (si se usa INPUT_PULLUP) o a 3.3V (si se usa resistencia pull-down).
    - **Resistencia Pull-down (1kΩ):** Conectada entre el pin GPIO y GND si no se usa el pull-up interno.
- **Motor DC (con Puente H o Transistores Cruzados):**
    - Terminales del motor a las salidas del Puente H.
    - Entradas del Puente H a pines GPIO del ESP32 (ej., GPIO16, GPIO17) para controlar la dirección.

    - Puente H alimentado por fuente de 12V.
- **Control de Triac (para Motor AC):**
    - **Optoacoplador MOC3021:**
        - Pin 1 (Ánodo del LED interno) a un pin GPIO del ESP32 (ej., GPIO4) a través de una resistencia limitadora de corriente (ej., 220Ω).
        - Pin 2 (Cátodo del LED interno) a GND del ESP32.
        - Pines 4 y 6 (Salida del fototriac) conectados al circuito de disparo del Triac.
    - **Triac (BT136):**
        - MT1 y MT2 (terminales principales) en serie con la carga (motor AC) y la línea de AC.
        - Gate (compuerta) conectado al Pin 4 del MOC3021 (generalmente con una resistencia de polarización) y Pin 6 del MOC3021 al MT2 del Triac. (La configuración exacta del disparo del triac puede variar ligeramente según la hoja de datos y la carga).

**6. RESULTADOS OBTENIDOS**

Los cuatro proyectos se implementaron con éxito, demostrando la viabilidad y robustez del ESP32 para diversas aplicaciones de control:

- **Servocontrol Preciso:** Los servomotores respondieron de manera precisa y consistente a las señales PWM generadas por el ESP32, alcanzando los ángulos deseados tanto con el control de un solo botón como con los dos botones para posiciones extremas.
- **Inversión de Giro de Motor DC:** El motor DC invirtió su sentido de giro de forma fiable cada 5 segundos, confirmando la correcta implementación de la lógica de temporización no bloqueante (millis()) y el control del Puente H.
- **Control Efectivo de Carga AC:** El circuito de triac y optoacoplador permitió el control seguro y eficiente del motor AC mediante una señal digital del ESP32, lo que valida la capacidad de la plataforma para interactuar con sistemas de alta potencia.
- **Funcionalidad Integrada:** La capacidad del ESP32 para gestionar simultáneamente diferentes tipos de actuadores y entradas (PWM, GPIO digital, temporización) fue claramente demostrada.

**7. CONCLUSIONES Y PERSPECTIVAS FUTURAS**

Lo que realmente aprendimos con el ESP32 en estos experimentos es que es un **verdadero todoterreno para controlar cosas**. Nos permitió desde mover un pequeño servo con un botón, hasta hacer que un motor girara solo, e incluso controlar un aparato conectado a la corriente de la casa de forma segura.

**I.T. ESCUELA INDUSTRIAL SUPERIOR**
**PEDRO DOMINGO MURILLO**
**CARRERA DE INFORMATICA INDUSTRIAL**




**SIM 300 SISTEMAS MICROPROCESADOS**

**INFORME PROYECTO**

# SISTEMA DE MONITOREO Y CONTROL AMBIENTAL PARA INVERNADERO BASADO EN ESP32

**ESTUDIANTE:**

1.-CORILA GUTIERREZ LUIS RODRIGO

FECHA: 12/JUNIO/2025

**LA PAZ – BOLIVIA**

El presente informe detalla el diseño e implementación de un sistema automatizado para el monitoreo y control de variables ambientales dentro de un invernadero, utilizando el microcontrolador ESP32 como cerebro central. El sistema integra múltiples sensores (humedad, temperatura ambiente y temperatura de suelo) para la adquisición continua de datos, y actuadores como un ventilador, un foco y un servomotor para la regulación de las condiciones. Además, se incorpora una pantalla LCD para la visualización local de los datos y una interfaz web alojada en el ESP32, accesible vía Wi-Fi, para el control remoto de ciertos elementos. Este proyecto demuestra una solución eficiente y escalable para la agricultura inteligente y la gestión ambiental controlada.

**1. OBJETIVO GENERAL**

Diseñar e implementar un sistema integral de **monitoreo y control ambiental automatizado** para un invernadero, aprovechando las capacidades del microcontrolador ESP32. El sistema deberá ser capaz de adquirir datos de sensores clave, activar actuadores de forma automática según umbrales predefinidos, proporcionar retroalimentación visual local a través de una pantalla LCD, y ofrecer control manual remoto mediante una interfaz web accesible vía Wi-Fi.

**2. MATERIALES Y COMPONENTES**

La implementación de este sistema requirió los siguientes componentes:

- **ESP32 DevKit:** Microcontrolador principal, encargado de la lógica de control, procesamiento de datos de sensores y gestión de la conectividad Wi-Fi.
- **Sensor de Humedad y Temperatura DHT11:** Para medir la humedad relativa y la temperatura ambiente del invernadero.
- **Sensor MAX6675 con Termocupla Tipo K:** Para una medición precisa de la temperatura, idealmente del suelo o de puntos específicos dentro del invernadero.
- **Pantalla LCD 16x2 con Módulo I2C:** Para la visualización local y en tiempo real de los datos ambientales y el estado del sistema. El módulo I2C simplifica las conexiones.
- **Módulo Relé de 5V:** Para el control seguro del foco AC, aislando el circuito de baja tensión del ESP32.
- **Transistor 2N2222:** Utilizado como interruptor electrónico para activar el ventilador y el relé, protegiendo los pines del ESP32.
- **Diodo Rectificador 1N4007:** Para proteger el transistor de picos de voltaje inverso generados por cargas inductivas como el relé.
- **Ventilador 12V:** Actuador para la regulación de la temperatura interna del invernadero (refrigeración/circulación de aire).
- **Servomotor SG90:** Para el control de una puerta, ventana o compuerta de ventilación del invernadero, permitiendo su apertura/cierre.
- **Fuentes de Alimentación (5V y 12V):** Para energizar el ESP32, la LCD, los servos, el relé y el ventilador.
- **Cables Dupont, Protoboard:** Para las conexiones y el prototipado del circuito.

## 3. FUNDAMENTO TEÓRICO

### 3.1 Adquisición de Datos de Sensores

- **DHT11:** Este sensor digital de bajo costo se comunica con el ESP32 a través de un protocolo de un solo cable. Proporciona lecturas de humedad relativa y temperatura ambiente, esenciales para el monitoreo de las condiciones del aire.
- **MAX6675 con Termocupla Tipo K:** El MAX6675 es un convertidor de analógico a digital (ADC) para termocuplas tipo K. Se comunica con el ESP32 usando el protocolo **SPI (Serial Peripheral Interface)**, ofreciendo lecturas de temperatura de alta precisión, especialmente útiles para medir la temperatura del suelo o puntos críticos.

### 3.2 Lógica de Control Automatizado

El ESP32 es el encargado de procesar los datos de los sensores. Se implementa una **lógica de control basada en umbrales**:

- Si la temperatura ambiente (del DHT11) supera un límite predefinido (ej., 30°C), o si la humedad relativa excede otro umbral (ej., 70%), el ventilador se activa automáticamente para ayudar a regular el ambiente.
- Esta lógica de control puede expandirse para incluir otras variables o secuencias más complejas.

### 3.3 Visualización y Feedback Local

- **Pantalla LCD 16x2 con I2C:** La comunicación **I2C (Inter-Integrated Circuit)** es un protocolo de comunicación serial que permite conectar múltiples dispositivos usando solo dos cables (SDA y SCL). El módulo I2C en la LCD simplifica enormemente el cableado al ESP32, permitiendo mostrar los valores de temperatura, humedad y el estado de los actuadores directamente en el lugar, sin necesidad de un dispositivo externo.

### 3.4 Control Remoto vía Web (Wi-Fi)

- **Servidor Web en ESP32:** Similar a proyectos anteriores, el ESP32 se configura como un servidor web que hospeda una página HTML. Esta página contiene elementos de interfaz (botones, sliders) que permiten al usuario interactuar con el sistema a través de Wi-Fi desde cualquier navegador.
- **Servomotor SG90 (Control de Puerta):** El servomotor se controla mediante **PWM**, como se explicó en informes previos. La interfaz web envía comandos al ESP32 para mover el servo a una posición de "abierto" (ej., 90°) o "cerrado" (ej., 0°), permitiendo la gestión remota de una entrada o salida del invernadero.
- **Foco AC (Control de Iluminación):** El control del foco se realiza a través del módulo relé, activado por un transistor NPN. La interfaz web proporciona un botón para encender/apagar el foco, permitiendo al usuario controlar la iluminación suplementaria del invernadero de forma remota.

## 4. DESARROLLO DEL PROYECTO

El proyecto se estructuró en las siguientes fases interconectadas:

1. **Adquisición de Datos de Sensores:**
    - Se configuraron las librerías y pines para la lectura continua del **DHT11**.

- Se implementó la comunicación **SPI** para el **MAX6675**, asegurando lecturas precisas de la termocupla.
- Los datos de ambos sensores se almacenaron en variables para su procesamiento.

2. **Lógica de Control del Ventilador:**
   - Se establecieron los **umbrales** para la temperatura y la humedad.
   - En el bucle principal del programa, se evaluaron continuamente las lecturas de los sensores. Si la temperatura superaba los 30°C O la humedad superaba el 70%, el pin GPIO conectado al transistor del ventilador se ponía en HIGH, activándolo. En caso contrario, el pin se ponía en LOW, apagándolo.
3. **Control Remoto Web (Foco y Servo):**
   - Se configuró el ESP32 para conectarse a la red Wi-Fi local y actuar como **servidor web**.
   - Se diseñó una **página HTML básica** con botones para "Abrir Puerta", "Cerrar Puerta" y "Encender/Apagar Foco".
   - Se definieron las **rutas HTTP** (endpoints) en el código del ESP32 para responder a las solicitudes de los botones. Al recibir una solicitud, el ESP32 activaba el pin PWM del servo o el pin GPIO del relé según correspondiera.
4. **Visualización en Pantalla LCD:**
   - Se inicializó la comunicación **I2C** con la pantalla LCD.
   - Los datos de temperatura, humedad y el estado de los actuadores (ventilador, foco, puerta) se formatearon y se mostraron en la pantalla LCD de forma periódica, proporcionando información instantánea al usuario.
5. **Conexiones de Actuadores:**
   - El **ventilador** se conectó a una fuente de 12V y su negativo a través de un transistor 2N2222, controlado por un GPIO del ESP32.
   - El **foco AC** se conectó al módulo relé, el cual a su vez fue activado por un transistor 2N2222 y un diodo 1N4007, gestionado por otro GPIO del ESP32.
   - El **servomotor SG90** se conectó directamente a un pin PWM del ESP32 y a la fuente de 5V.

**5. CONEXIONES PRINCIPALES**

- **Sensor DHT11:**
  - VCC a **3.3V** del ESP32.
  - Pin DATA a un pin GPIO digital del ESP32 (ej., GPIO4).
  - GND a **GND** del ESP32.
- **Sensor MAX6675 (con Termocupla):**
  - VCC a **3.3V** del ESP32.

    - GND a **GND** del ESP32.
    - **SCK** (Clock) a GPIO del ESP32 (ej., GPIO18 - SCK por defecto).
    - **CS** (Chip Select) a GPIO del ESP32 (ej., GPIO5).
    - **SO** (MISO - Master In Slave Out) a GPIO del ESP32 (ej., GPIO19 - MISO por defecto).
- **Pantalla LCD 16x2 con Módulo I2C:**
    - VCC a **5V** (directamente del ESP32 o de una fuente externa de 5V).
    - GND a **GND** del ESP32.
    - SDA a **GPIO21** del ESP32.
    - SCL a **GPIO22** del ESP32.
- **Ventilador 12V (con Transistor 2N2222):**
    - Positivo del ventilador a **12V** de la fuente externa.
    - Negativo del ventilador al colector del transistor 2N2222.
    - Emisor del 2N2222 a **GND** común.
    - Base del 2N2222 a un pin GPIO del ESP32 (ej., GPIO16) a través de una resistencia de 1kΩ.
- **Foco AC (con Módulo Relé 5V y Transistor 2N2222):**
    - Conexiones del relé con el foco AC y la línea eléctrica idénticas a las descritas en informes anteriores.
    - Pin de control (IN) del relé al colector del transistor 2N2222.
    - Emisor del 2N2222 a **GND** común.
    - Base del 2N2222 a un pin GPIO del ESP32 (ej., GPIO17) a través de una resistencia de 1kΩ.
    - **Diodo 1N4007** en paralelo con la bobina del relé para protección.
- **Servomotor SG90:**
    - Cable de Señal (Amarillo/Naranja) a un pin PWM del ESP32 (ej., GPIO13).
    - Cable VCC (Rojo) a **5V** de la fuente externa (es importante alimentar el servo con su propia fuente si es posible, para evitar sobrecargar el ESP32).
    - Cable GND (Marrón/Negro) a **GND** común.

**6. RESULTADOS OBTENIDOS**

El sistema automatizado para el invernadero se implementó con éxito, demostrando un funcionamiento fiable y una integración eficiente de hardware y software:

- **Monitoreo Preciso:** La lectura de los sensores DHT11 y MAX6675 fue continua y precisa, proporcionando datos fiables de temperatura ambiente y de suelo/termoacoplador.

- **Automatización Efectiva:** El ventilador se activó automáticamente y de manera consistente cuando las condiciones ambientales (temperatura o humedad) superaron los umbrales predefinidos, demostrando la eficacia de la lógica de control automatizado.
- **Control Remoto Funcional:** La interfaz web alojada en el ESP32 permitió un control manual fluido y responsivo del foco AC y del servomotor (puerta del invernadero). Los comandos desde el navegador se interpretaron y ejecutaron correctamente en el ESP32.
- **Visualización Local Clara:** La pantalla LCD 16x2 mostró de forma clara y legible los datos ambientales en tiempo real (temperatura, humedad) y el estado de los actuadores, ofreciendo una retroalimentación instantánea sin necesidad de dispositivos externos.
- **Integración Exitosa:** El proyecto demostró una excelente sinergia entre los diferentes módulos (sensores, actuadores, pantalla, conectividad Wi-Fi), creando un sistema coherente y funcional.

**7. CONCLUSIONES**

Este proyecto nos demostró que el ESP32 es un verdadero **cerebro para el invernadero** del futuro (o de hoy). Pudimos crear un sistema que no solo "siente" el ambiente (temperatura y humedad), sino que también "reacciona" automáticamente, por ejemplo, encendiendo un ventilador cuando hace demasiado calor.

Lo más genial es que tenemos todo bajo control: vemos los datos en una pantallita ahí mismo en el invernadero, y si queremos abrir una puerta o encender una luz, lo hacemos **cómodamente desde nuestro celular vía Wi-Fi**. Es una combinación perfecta de automatización inteligente y la posibilidad de intervenir manualmente cuando sea necesario. Esto abre las puertas a una agricultura más eficiente, donde las plantas pueden crecer en su ambiente ideal, ¡y nosotros solo tenemos que preocuparnos por cosechar!

**PINES USADOS DEL ESP32**

Los pines del ESP32 que debes usar, según el código que proporcionaste y las conexiones descritas en el informe, son los siguientes:

**Pines Utilizados en tu Proyecto de Invernadero Automatizado:**

- **Sensor DHT11 (Humedad y Temperatura Ambiente):**
    - **GPIO 4** (#define DHTPIN 4)
- **Sensor MAX6675 con Termocupla Tipo K (Temperatura de Suelo/Precisa):**
    - **GPIO 18** (#define MAX6675_SCK_PIN 18) - Pin SCK (Clock)
    - **GPIO 5** (#define MAX6675_CS_PIN 5) - Pin CS (Chip Select)
    - **GPIO 19** (#define MAX6675_SO_PIN 19) - Pin SO (MISO - Master In Slave Out)
- **Pantalla LCD 16x2 con Módulo I2C:**
    - **GPIO 21** (SDA - Data Line)
    - **GPIO 22** (SCL - Clock Line) *(Estas son las configuraciones I2C estándar para el ESP32)*

- **LED de Alerta de Temperatura:**
    - **GPIO 25** (const int TEMP_LED_PIN = 25;)
- **LED de Alerta de Humedad:**
    - **GPIO 26** (const int HUMIDITY_LED_PIN = 26;)
- **Servomotor SG90 (para la puerta/compuerta):**
    - **GPIO 27** (const int SERVO_PIN = 27;) - Este pin se usará para la señal PWM.

**Pines Adicionales Recomendados (para Ventilador y Foco/Relé):**

Tu informe menciona el uso de un **ventilador de 12V** y un **relé de 5V para el foco AC**, ambos controlados por transistores 2N2222. Aunque el código no asigna pines explícitamente para ellos, puedes usar estos GPIOs que son adecuados para salidas digitales:

- **Ventilador 12V (controlado por Transistor 2N2222):**
    - Podrías usar **GPIO 16** o **GPIO 17**. Por ejemplo, **GPIO 16**.
- **Foco AC (controlado por Relé 5V y Transistor 2N2222):**
    - Podrías usar **GPIO 17** o **GPIO 23**. Por ejemplo, **GPIO 17**.

**Consideraciones Generales sobre los Pines del ESP32:**

- **Flexibilidad:** El ESP32 es muy flexible y la mayoría de sus pines GPIO pueden usarse para diferentes funciones (digital I/O, PWM, ADC, etc.), excepto algunos que tienen funciones específicas de arranque o flash.
- **Pines de Propósito General (GPIOs):** Los pines que no están mencionados con una función específica (como ADC, SPI, I2C predeterminados) pueden usarse como entradas/salidas digitales.
- **Pines a Evitar o Usar con Precaución durante el Arranque:** Algunos pines (como GPIO0, GPIO2, GPIO4, GPIO5, GPIO12, GPIO15) tienen estados específicos al momento del arranque o se usan para la programación, por lo que es mejor evitar conectarles cargas que puedan interferir con este proceso, o al menos ser consciente de su comportamiento. Para este proyecto, los pines seleccionados son generalmente seguros. Basándome en el código que proporcionaste y el informe del proyecto, aquí tienes un desglose de los pines del ESP32 que estás utilizando o que necesitarías usar:

**Pines Utilizados Directamente en tu Código:**

- **Sensor DHT11 (Humedad y Temperatura Ambiente):**
    - DHTPIN 4 (GPIO 4)
- **Sensor MAX6675 con Termocupla Tipo K (Temperatura Específica/Suelo):**
    - MAX6675_SCK_PIN 18 (GPIO 18) - Pin de reloj (SCK)
    - MAX6675_CS_PIN 5 (GPIO 5) - Pin de selección de chip (CS)
    - MAX6675_SO_PIN 19 (GPIO 19) - Pin de datos (SO/MISO)

- **Pantalla LCD 16x2 con Módulo I2C:**
    - SDA: **GPIO 21**
    - SCL: **GPIO 22**
    - *(Estos son los pines I2C por defecto en la mayoría de las placas ESP32 y son los que usas implícitamente al inicializar LiquidCrystal_I2C lcd(LCD_ADDR, LCD_COLS, LCD_ROWS); sin especificar pines en lcd.begin().)*
- **LED de Alerta de Temperatura:**
    - TEMP_LED_PIN 25 (GPIO 25)
- **LED de Alerta de Humedad:**
    - HUMIDITY_LED_PIN 26 (GPIO 26)
- **Servomotor SG90 (Puerta):**
    - SERVO_PIN 27 (GPIO 27) - Pin PWM

**Pines Adicionales Sugeridos (Basados en el Informe y Componentes Restantes):**

Tu informe menciona el uso de un **ventilador de 12V** y un **relé para un foco AC**. Aunque no están definidos explícitamente con pines en tu código, necesitarás conectar estos a GPIOs para controlarlos.

Considerando las buenas prácticas y los pines disponibles, podrías usar:

- **Ventilador 12V (controlado por Transistor 2N2222):**
    - **GPIO 16** (Este es un buen pin de propósito general para salida).
- **Relé 5V para Foco AC (controlado por Transistor 2N2222 y Diodo 1N4007):**
    - **GPIO 17** (También es un buen pin de propósito general para salida).

**Resumen de pines recomendados para tu proyecto:**

| Componente | Pin ESP32 | Función Específica / Nota |
|---|---|---|
| **Sensor DHT11** | GPIO 4 | Entrada de datos (Humedad y Temp. ambiente) |
| **MAX6675 (SCK)** | GPIO 18 | Comunicación SPI (Reloj) |
| **MAX6675 (CS)** | GPIO 5 | Comunicación SPI (Selección de Chip) |
| **MAX6675 (SO/MISO)** | GPIO 19 | Comunicación SPI (Datos) |
| **LCD I2C (SDA)** | GPIO 21 | Comunicación I2C (Datos) |
| **LCD I2C (SCL)** | GPIO 22 | Comunicación I2C (Reloj) |
| **LED Alerta Temp.** | GPIO 25 | Salida digital |
| **LED Alerta Humedad** | GPIO 26 | Salida digital |

| | | |
|---|---|---|
| **Servomotor SG90** | GPIO 27 | Salida PWM (Control de posición) |
| **Ventilador 12V (Control)** | GPIO 16 | Salida digital (via Transistor 2N2222) |
| **Relé 5V para Foco AC (Control)** | GPIO 17 | Salida digital (via Transistor 2N2222 y Diodo 1N4007) |

**Consideraciones importantes al elegir pines en ESP32:**

- **Pines de Boot (Strapping Pins):** Algunos pines (GPIO0, GPIO2, GPIO4, GPIO5, GPIO12, GPIO15) tienen funciones especiales durante el arranque. Si usas estos pines, asegúrate de que su estado al encender el ESP32 no interfiera con el proceso de carga del programa (por ejemplo, si un botón los mantiene en LOW y se usan para el modo de flasheo). Los pines seleccionados arriba son generalmente seguros.
- **Pines de Entrada Solamente:** Los GPIOs 34, 35, 36, 39 son solo para entrada y no tienen pull-ups/pull-downs internos. No puedes usarlos como salidas.
- **Conflictos con Flash SPI:** Los GPIOs 6 a 11 (y a veces 16-17) están conectados a la memoria flash SPI interna del ESP32 y no se recomienda usarlos para otras funciones, ya que pueden causar problemas de funcionamiento o impedir el arranque. Los pines sugeridos (16 y 17) para el ventilador y relé son generalmente seguros, pero siempre es bueno estar atento a posibles conflictos con módulos que usan SPI en tu placa específica si se diera el caso.
- **Pines ADC2 y Wi-Fi:** Los pines ADC2 (GPIO 2, 4, 12, 13, 14, 15, 25, 26, 27) pueden tener problemas para leer valores analógicos si la Wi-Fi está activa. En tu caso, los sensores DHT11 y MAX6675 son digitales, y el servo es PWM, así que esto no es un problema directo, pero es algo a tener en cuenta para futuros sensores analógicos.
- **Alimentación:** Recuerda siempre conectar la alimentación (3.3V o 5V según el componente) y las conexiones a tierra (GND) correctamente. Para componentes que demandan mucha corriente (como el ventilador o el relé que a su vez activa el foco AC), es crucial usar una fuente de alimentación externa y no depender únicamente de la del ESP32.

